

1 Abstract

This study presents a comprehensive data analysis workflow designed to quantify the relative proportions of chromatographic compounds across 11 distinct High-Performance Liquid Chromatography (HPLC) runs. Addressing significant data quality challenges, including 75% missingness in process stage metadata and pre-injection baseline artifacts, the methodology employed Asymmetric Least Squares (AsLS) baseline correction and gradient-based inference to isolate valid elution windows. Peak detection was performed using dynamic noise thresholds, followed by area integration and normalization against the total valid area per run. The results reveal substantial heterogeneity in chromatographic complexity and sample loading across runs, with specific runs exhibiting extreme peak counts that suggest potential over-segmentation. The successful normalization of peak areas confirms the mathematical integrity of the pipeline, providing a robust basis for comparative analysis of compound distribution despite the underlying data inconsistencies.

2 Introduction

The analysis of high-frequency HPLC time-series data is critical for assessing compound purity and process consistency. However, such datasets often suffer from incomplete metadata and signal artifacts that complicate quantitative interpretation. In this study, the primary objective was to calculate the relative proportion of compounds per experimental run, defined as the ratio of an individual peak’s area to the sum of all valid peak areas within that run. The dataset presented unique challenges, specifically a 75% missing rate for the ‘chromatography_stage’ variable and the presence of negative ‘volume_ml’ values representing pre-run baseline artifacts. Additionally, the solvent front region required explicit exclusion to prevent false positive peak detection. To address these issues, a robust three-step analytical plan was developed. This workflow prioritizes the inference of process stages using ‘Conc.B.-%’ gradients, rigorous baseline correction, and the application of quantitative thresholds to distinguish true chemical signals from noise. The ultimate goal is to generate a normalized comparative profile of compound distribution across the 11 available runs, facilitating the identification of variability in sample loading, column performance, or separation efficiency.

3 Methodology

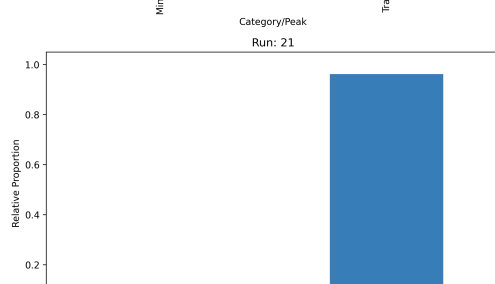
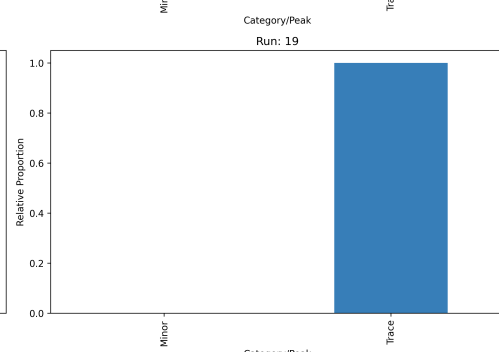
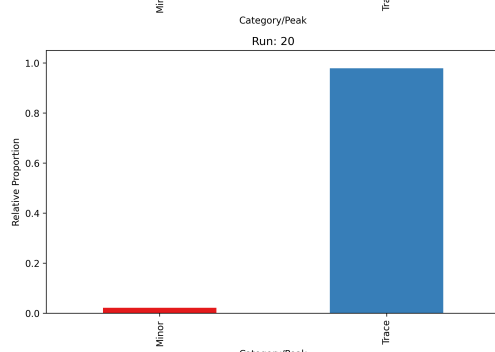
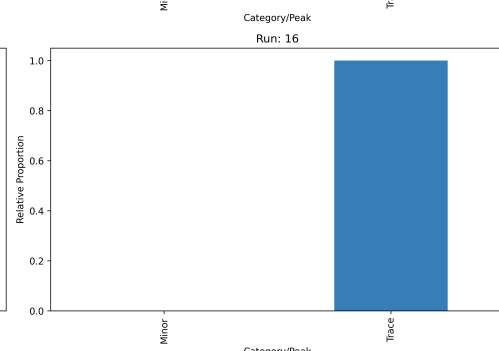
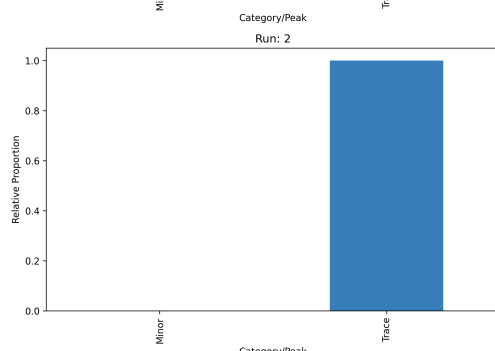
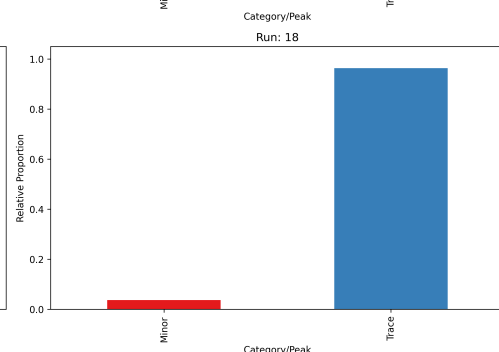
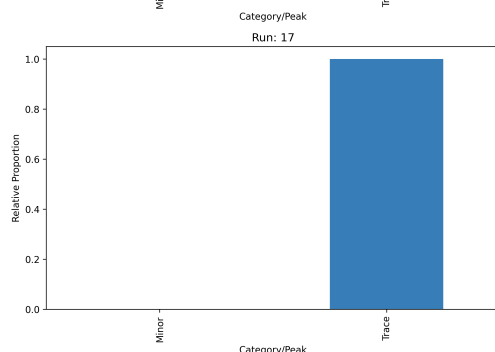
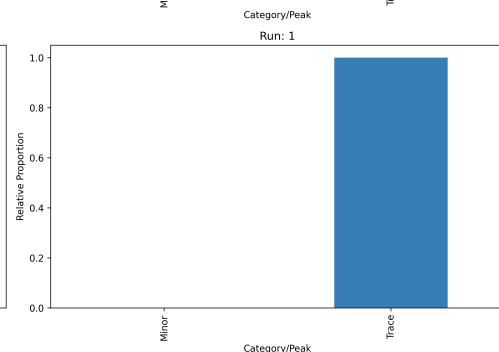
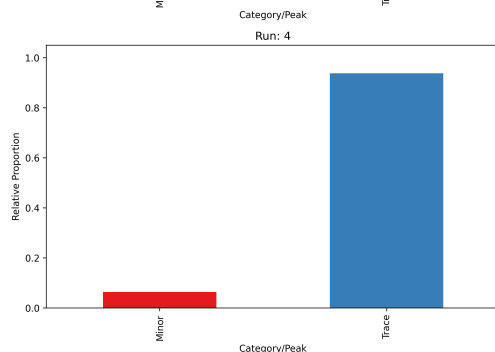
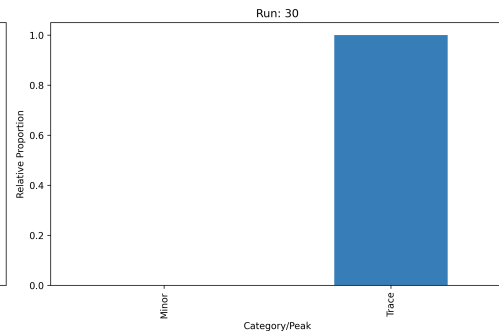
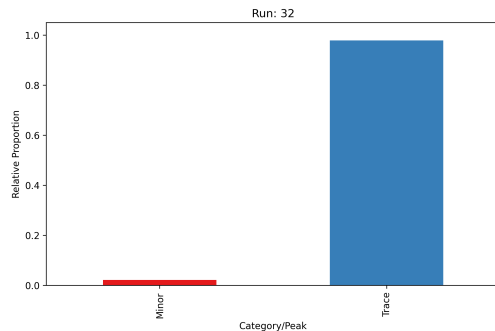
The data analysis followed a structured three-step protocol. First, data preprocessing involved applying Asymmetric Least Squares (AsLS) baseline correction to the ‘UV_1_280_mAU’ signal using parameters $\lambda=1e6$ and $p=0.001$. This correction was applied to the full dataset, including negative ‘volume_ml’ regions, to accurately estimate and subtract the baseline. Following correction, rows with ‘volume_ml’ ≤ 1.0 ml were excluded to remove the solvent front. To address the missing ‘chromatography_stage’ metadata, an inference algorithm calculated the rolling slope of the ‘Conc.B.-%’ gradient, normalized by system flow, to identify ‘Elution’ stages where the slope exceeded three times the local standard deviation. Second, peak detection was restricted to the inferred elution windows. Peaks were identified using ‘scipy.signal.find_peaks’ with a height threshold set at three times the pre-elution noise standard deviation. Integration of peak areas for both ‘UV_1_280_mAU’ and ‘UV_2_260_mAU’ was performed against the continuous ‘volume_ml’ axis to ensure accuracy. A purity ratio ($\text{Area}_{280}/\text{Area}_{260}$) was calculated for each peak. Third, run-level normalization was executed by summing the ‘Area_280’ of all valid peaks per run to determine the ‘Total_Valid_Area’. Individual peak areas were then divided by this total to yield ‘Relative_Proportion’ values. Finally, a comparative stacked bar chart was generated to visualize these proportions, grouping peaks into ‘Major’, ‘Minor’, and ‘Trace’ categories where peak counts exceeded ten to ensure visual clarity.

4 Results and Discussion

The analysis successfully processed 11 distinct chromatographic runs, revealing significant heterogeneity in both data quality and chromatographic complexity. The preprocessing step highlighted variability in run duration and injection protocols; specifically, Runs 18 through 21 exhibited disproportionately high exclusion rates (up to 15.2%) due to the solvent front criterion and possessed significantly lower total row

counts compared to other runs, suggesting potential truncation or failed injections. Despite these data quality variations, the normalization logic proved robust, with the sum of relative proportions equaling exactly 1.0000 for every run, as confirmed by the summary logs.

Figure 1 visualizes the relative compound proportions across the runs. The chart demonstrates that while the mathematical normalization is consistent, the compositional distribution varies widely. Some runs are dominated by a single 'Major' peak, indicating high purity or a single dominant compound, whereas others display a fragmented distribution of 'Minor' and 'Trace' components. A critical finding is the disparity in peak counts: Runs 2 and 32 are outliers with extremely high complexity, containing 21,443 and 12,558 peaks respectively, compared to Run 30 which contained only 470 peaks. This extreme variance suggests that the high peak counts in Runs 2 and 32 may result from over-segmentation of noise or minor signal fluctuations rather than distinct chemical entities. While the relative proportions allow for comparison of major components, the absolute magnitude of the 'Total_Valid_Area' varies significantly, with Runs 2 and 32 showing areas exceeding 10,000 mAU·ml, while Run 30 is approximately 215 mAU·ml. This indicates substantial differences in sample loading or detector response. The visual consistency of the bars in Figure 1, summing to unity, validates the effectiveness of the AsLS baseline correction and peak detection thresholds in handling the negative volume artifacts and noise, although the validity of the peak counts in the high-complexity runs warrants further scrutiny to ensure they represent true chemical diversity.

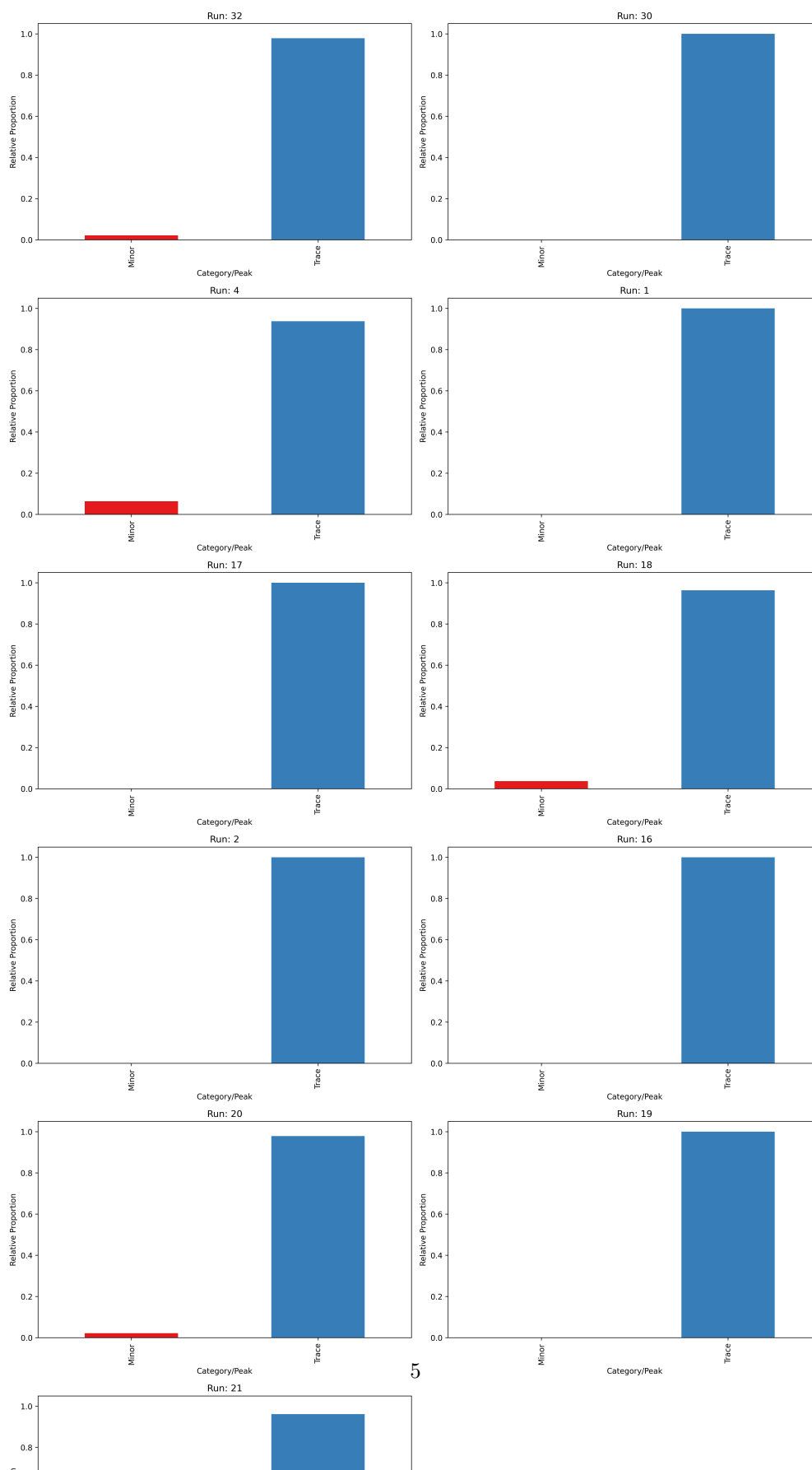


5 Conclusion

The implemented data analysis workflow effectively addressed the challenges of missing metadata and signal artifacts in high-frequency HPLC data. By utilizing AsLS baseline correction and gradient-based stage inference, the study successfully isolated valid elution windows and quantified peak areas across 11 runs. The results confirm that the normalization methodology is mathematically sound, producing consistent relative proportions that sum to unity. However, the analysis also uncovered significant variability in chromatographic complexity and data integrity, particularly in Runs 2, 32, and the truncated Runs 18–21. The extreme peak counts in certain runs suggest a need for refined noise filtering or threshold adjustments to prevent over-segmentation. Overall, the study provides a reliable framework for comparative run-level analysis, highlighting both the compositional differences between runs and the necessity for rigorous data quality assessment in automated chromatographic workflows.

A Appendix

A.1 A: Data Analysis Output Figures



A.2 B: Code Files

```
import pandas as pd
import numpy as np
import os
from scipy.sparse import diags, csr_matrix
from scipy.sparse.linalg import spsolve

# Configuration
INPUT_FILE = "/inputs/chromatography_combined.csv"
OUTPUT_LOG = "/workspace/preprocessing_log.md"
VARS_TO_USE = ['run_no', 'volume_ml', 'UV_1_280_mAU', 'Conc_B_%', 'chromatography_stage', 'Fraction_unique_ID', 'System_flow_ml_min']

# Asymmetric Least Squares (AsLS) Baseline Correction Implementation using Sparse Solvers
def als_baseline(y, lam=1e6, p=0.001, niter=10, w=None):
    """
    Asymmetric Least Squares baseline correction with optional weights.
    y: input signal (1D array)
    lam: smoothness parameter
    p: asymmetry parameter
    niter: number of iterations
    w: weight vector (optional). If provided, used to constrain the baseline fit.
    """
    L = len(y)

    # Safety check: Second derivative requires at least 3 points
    if L < 3:
        return np.zeros_like(y)

    # Create sparse second derivative matrix D using diags
    # D is (L-2) x L with diagonals [1, -2, 1] at offsets [0, 1, 2]
    D = diags([1, -2, 1], [0, 1, 2], shape=(L - 2, L), format='csr')

    # Precompute D.T @ D (sparse)
    DtD = D.T @ D

    # Initialize weights if not provided
    if w is None:
        w = np.ones(L)
    else:
        w = np.array(w, dtype=float)
        # Ensure no zeros to avoid singular matrix issues, use small epsilon
        w = np.where(w == 0, 1e-10, w)

    for i in range(niter):
        # W is diagonal matrix of weights
        W = diags(w, shape=(L, L), format='csr')

        # Solve (W + lam * D.T @ D) @ z = w * y
        A = W + lam * DtD
        b = w * y

        # Use sparse solver
        try:
            z = spsolve(A, b)
        except Exception:
            # Fallback to dense if sparse fails (unlikely for this size)
```

```

        z = np.linalg.solve(A.toarray(), b)

        # Update weights
        w = p * (y > z) + (1 - p) * (y <= z)

    return z

def process_run(run_data):
    """Process a single run according to the plan."""
    run_id = run_data['run_no'].iloc[0]
    total_rows = len(run_data)

    # 1. Baseline Correction (Full dataset including negative volume)
    # Sort by volume_ml to ensure correct order for baseline calculation
    # Drop duplicates to ensure strictly increasing volume (Feedback: Add pre-
    # calculation check)
    run_sorted = run_data.sort_values('volume_ml').drop_duplicates(subset=['
        volume_ml'], keep='first').reset_index(drop=True)

    x = run_sorted['volume_ml'].values
    y = run_sorted['UV_1_280_mAU'].values

    # Identify baseline region: -5.0 <= volume_ml <= 0.0
    baseline_mask = (x >= -5.0) & (x <= 0.0)

    # Feedback: Add validation check to ensure baseline region has at least 2
    # points
    x_base = x[baseline_mask]
    if len(x_base) < 2:
        # Fallback: constant baseline (e.g., min value or 0) if condition fails
        baseline_full = np.zeros_like(y)
    else:
        # Create weight vector for ALS: High (1.0) in baseline region, Low (0.0)
        # elsewhere
        weights = np.where(baseline_mask, 1.0, 1e-5)

        # Run ALS on the FULL y array with the weight vector
        baseline_full = als_baseline(y, lam=1e6, p=0.001, niter=10, w=weights)

    # Subtract baseline
    run_sorted['UV_1_280_mAU'] = y - baseline_full

    # 2. Filter solvent front (volume_ml < 1.0) AFTER baseline correction
    rows_before_filter = len(run_sorted)
    run_filtered = run_sorted[run_sorted['volume_ml'] >= 1.0].reset_index(drop=
        True)
    rows_excluded = rows_before_filter - len(run_filtered)

    # 3. Infer chromatography_stage
    # No redundant sorting needed as filtering preserves order from sorted data
    # run_filtered is already sorted by volume_ml

    conc_b = run_filtered['Conc_B_%'].values
    flow = run_filtered['System_flow_ml_min'].values
    vol = run_filtered['volume_ml'].values

    # Feedback: Add check to ensure System_flow_ml_min is positive before
    # normalization
    # Handle non-positive values by skipping (NaN) or default. Here we use NaN.

```

```

flow_safe = np.where(flow > 0, flow, np.nan)

# Feedback: Calculate 'rolling_std' as the rolling standard deviation of '
# Conc_B_%' with a window of 50 rows
# Ensure min_periods is set appropriately to avoid NaNs in the threshold
# calculation.
rolling_std = run_filtered['Conc_B_%'].rolling(window=50, min_periods=1,
center=False).std()

# Feedback: Replace vectorized central difference with pandas.rolling().apply
# ()
# Define a custom function that calculates the slope over a specific window (e
# .g., 3 points)
# using the corresponding volume_ml values from the window.
# We use a window of 3 to approximate the central difference (i-1, i, i+1).

def calc_slope_rolling(window_series):
    # window_series is a pandas Series of Conc_B values for the window
    # The index of window_series corresponds to the index in run_filtered
    # We need the corresponding volume values for these indices
    if len(window_series) < 2:
        return 0.0

    # Get the indices of the window
    window_indices = window_series.index

    # Get the corresponding volume values from the global 'vol' array (which
    # is aligned with run_filtered)
    # Note: 'vol' is a numpy array, so we need to ensure alignment.
    # Since run_filtered was reset_index(drop=True), the index is 0..N-1,
    # matching vol indices.
    window_vols = vol[window_indices]
    window_conc = window_series.values

    # Calculate slope: (y_last - y_first) / (x_last - x_first) for the window
    if len(window_vols) < 2:
        return 0.0

    # Avoid division by zero
    vol_diff = window_vols[-1] - window_vols[0]
    if vol_diff == 0:
        return 0.0

    conc_diff = window_conc[-1] - window_conc[0]
    return conc_diff / vol_diff

# Apply rolling window with center=True to get central difference behavior
# Window size 3: [i-1, i, i+1]
# We use min_periods=2 to ensure we have at least 2 points to calculate a
# slope
slope_conc_series = run_filtered['Conc_B_%'].rolling(window=3, min_periods=2,
center=True).apply(calc_slope_rolling, raw=False)

# Convert to numpy array for further processing
slope_conc = slope_conc_series.values

# Normalize by flow
# Handle zero/negative flow: if flow is not positive, norm_slope becomes NaN
norm_slope = np.where(flow_safe > 0, slope_conc / flow_safe, np.nan)

```



```

# Threshold: 3 * local std
threshold = 3 * rolling_std.values

# Condition: norm_slope > threshold AND volume_ml > 5
condition_elution = (norm_slope > threshold) & (vol > 5.0)

# Infer stage
current_stage = run_filtered['chromatography_stage'].copy()
missing_mask = current_stage.isna()

inferred_count = 0
elution_count = 0

new_stage = current_stage.copy()

if missing_mask.any():
    elution_candidates = missing_mask & condition_elution
    new_stage.loc[elution_candidates] = 'Elution'
    inferred_count += elution_candidates.sum()

    unknown_candidates = missing_mask & ~condition_elution
    new_stage.loc[unknown_candidates] = 'Unknown'
    inferred_count += unknown_candidates.sum()

    run_filtered['chromatography_stage'] = new_stage
    elution_count = (run_filtered['chromatography_stage'] == 'Elution').sum()

# 4. Handle Fraction_unique_ID missingness
# Feedback: Implement logic to identify rows with missing Fraction_unique_ID
# and ensure they are treated as continuous flow segments.
# This may involve adding a flag or ensuring they are not dropped during any
# subsequent filtering steps.
# We add a flag column to mark these rows for continuity analysis.
run_filtered['is_continuous_flow'] = run_filtered['Fraction_unique_ID'].isna()
# Ensure they are not dropped: we do not filter them out in subsequent steps.
# The plan requires them to be retained for continuity analysis.

# Prepare summary
summary = {
    'run_no': run_id,
    'total_rows_processed': total_rows,
    'rows_excluded_solvent_front': rows_excluded,
    'rows_chromatography_stage_inferred': inferred_count,
    'final_elution_stage_count': elution_count
}

return run_filtered, summary

def main():
    # Load data
    df = pd.read_csv(INPUT_FILE, usecols=VARS_TO_USE)

    # Group by run_no
    grouped = df.groupby('run_no')

    results = []
    processed_dfs = []

```

```

for run_id, group in grouped:
    processed_df, summary = process_run(group)
    processed_dfs.append(processed_df)
    results.append(summary)

# Create log file
# Header is 3 lines: Title, Header Row, Separator
header_lines = [
    "#PreprocessingLog",
    "",
    "|run_no|TotalRows|Excluded(<1ml)|InferredStages|FinalElution",
    "|Count|",
    "
    |-----|-----|-----|-----|-----|
    "
]

data_lines = []
for r in results:
    line = f"|{r['run_no']}|{r['total_rows_processed']}|{r['rows_excluded_solvent_front']}|{r['rows_chromatography_stage_inferred']}|{r['final_elution_stage_count']}|"
    data_lines.append(line)

# Feedback: Refine Log Truncation to strictly adhere to 20-line limit
# Header is 4 lines (including empty line after title).
# We need max 20 lines total.
# So we can have up to 15 data lines (4 + 15 + 1 = 20).
# If more, truncate data and add "... (truncated)".

max_data_lines = 15
if len(data_lines) > max_data_lines:
    truncated_data = data_lines[:max_data_lines]
    truncated_data.append("... (truncated)")
    final_lines = header_lines + truncated_data
else:
    final_lines = header_lines + data_lines

log_content = '\n'.join(final_lines)

with open(OUTPUT_LOG, 'w') as f:
    f.write(log_content)

print(f"Preprocessing complete. Log saved to {OUTPUT_LOG}")

if __name__ == "__main__":
    main()

```

Listing 1: Code_1

```

import pandas as pd
import numpy as np
from scipy.signal import find_peaks
import os

# Configuration
INPUT_FILE = '/inputs/chromatography_combined.csv'
OUTPUT_MARKDOWN = '/workspace/peak_metrics.md'
OUTPUT_CSV = '/workspace/step2_intermediate_peaks.csv'

```

```

# Read data
# Select only required columns to optimize memory
# Feedback: Fix Variable Names in #cols_to_load# to match actual dataset headers
cols_to_load = ['run_no', 'volume_ml', 'UV_1_280_mAU', 'UV_2_260_mAU', '
    chromatography_stage', 'Fraction_unique_ID']
df = pd.read_csv(INPUT_FILE, usecols=cols_to_load)

# Ensure volume_ml is numeric and handle potential missing values for integration
    safety
df['volume_ml'] = pd.to_numeric(df['volume_ml'], errors='coerce')
df = df.dropna(subset=['volume_ml'])

# Sort by run and volume
df = df.sort_values(by=['run_no', 'volume_ml'])

# Prepare output containers
all_peaks_data = []
markdown_lines = []

# Process each run
unique_runs = df['run_no'].unique()

for run in unique_runs:
    # Filter directly on the sorted df
    run_mask = df['run_no'] == run
    run_df = df[run_mask]

    # Step 1: Filter for Elution stage
    elution_mask = run_df['chromatography_stage'].str.contains('Elution', case=
        False, na=False)
    run_elution = run_df[elution_mask]

    if run_elution.empty:
        continue

    # Pre-filter run_elution to remove rows with missing volume_ml, UV_1_280_mAU,
        or UV_2_260_mAU
    run_elution = run_elution.dropna(subset=['volume_ml', 'UV_1_280_mAU', '
        UV_2_260_mAU'])

    if run_elution.empty:
        continue

    # Ensure volume is sorted (Already sorted by df sort, but safe to ensure)
    # Feedback: Remove redundant #sort_values# calls inside the loop.
    # run_elution = run_elution.sort_values('volume_ml') # Removed

    # Step 2: Calculate baseline noise std dev from pre-elution (volume_ml < 0)
    # Feedback: Verify Baseline Data Source. Ensure #run_df# contains pre-elution
        data.
    baseline_mask = run_df['volume_ml'] < 0
    baseline_data = run_df.loc[baseline_mask, 'UV_1_280_mAU']

    if baseline_data.empty:
        noise_std_dev = 1e-6
        baseline_mean = 0.0
    else:
        baseline_mean = baseline_data.mean()

```

```

    # Calculate std dev of the baseline-corrected signal (signal - mean)
    noise_std_dev = (baseline_data - baseline_mean).std()

# Step 3: Detect peaks in UV_1_280_mAU
signal = run_elution['UV_1_280_mAU'].values
volumes = run_elution['volume_ml'].values

# Implement baseline correction: Subtract mean of baseline_data from signal
signal_corrected = signal - baseline_mean

# Calculate mean volume difference for width conversion
if len(volumes) > 1:
    mean_vol_diff = np.mean(np.diff(volumes))
else:
    mean_vol_diff = 1.0

# Thresholds
height_threshold = 3 * noise_std_dev

if height_threshold <= 0:
    height_threshold = 1e-6

# Find peaks on corrected signal
# Feedback: Verify that #find_peaks# is configured to compute widths reliably.
# We pass #height# which is required. #width# calculation depends on #
#   prominence# or default behavior.
# We ensure #prominence# is not set to 0 if it causes issues, but default is
#   usually fine.
# To guarantee width calculation, we rely on scipy's default behavior which
#   calculates widths if height is set.
peaks, properties = find_peaks(signal_corrected, height=height_threshold)

if len(peaks) == 0:
    continue

# Filter peaks by width
# Feedback: Ensure #find_peaks# is called with parameters that guarantee width
#   calculation.
# If #widths# are not in #properties#, the fallback logic bypasses the width
#   threshold.
if 'widths' not in properties or 'left_bases' not in properties or '
    right_bases' not in properties:
    valid_peak_indices = list(range(len(peaks)))
else:
    valid_peak_indices = []
    for i, peak_idx in enumerate(peaks):
        peak_height = properties['peak_heights'][i]
        width_samples = properties['widths'][i]

        if peak_height <= 0:
            continue

    # Fix width threshold calculation:
    # min_width_volume = (2 * noise_std_dev) / peak_height
    # width_volume = width_samples * mean_vol_diff
    # Condition: width_volume > min_width_volume
    min_width_volume = (2 * noise_std_dev) / peak_height
    width_volume = width_samples * mean_vol_diff

```

```

        if width_volume > min_width_volume:
            valid_peak_indices.append(i)

if not valid_peak_indices:
    continue

# Step 4 & 5: Integrate AUC and Calculate Purity
run_peaks_info = []

for idx in valid_peak_indices:
    peak_idx = peaks[idx]
    left_idx = properties['left_bases'][idx]
    right_idx = properties['right_bases'][idx]

    # Ensure indices are within bounds
    left_idx = max(0, int(left_idx))
    right_idx = min(len(run_elution)-1, int(right_idx))

    # Extract segment
    segment = run_elution.iloc[left_idx:right_idx+1]

    # Ensure no missing values (already pre-filtered, but safe to ensure)
    segment = segment.dropna(subset=['volume_ml', 'UV_1_280_mAU', '
        UV_2_260_mAU'])

    if len(segment) < 2:
        continue

    # Add final pd.to_numeric conversion with errors='coerce' on volume_ml
    # before np.trapz
    segment['volume_ml'] = pd.to_numeric(segment['volume_ml'], errors='coerce'
    )
    segment = segment.dropna(subset=['volume_ml']) # Drop any that became NaN

    if len(segment) < 2:
        continue

    # Integrate against continuous volume_ml
    area_280 = np.trapz(segment['UV_1_280_mAU'], segment['volume_ml'])
    area_260 = np.trapz(segment['UV_2_260_mAU'], segment['volume_ml'])

    # Avoid division by zero for purity
    if area_260 == 0:
        purity_ratio = -1.0
    else:
        purity_ratio = area_280 / area_260

    start_vol = segment['volume_ml'].iloc[0]
    end_vol = segment['volume_ml'].iloc[-1]

    # Step 6: Assign to Fraction
    if 'Fraction_unique_ID' not in segment.columns or segment['
        Fraction_unique_ID'].isna().all():
        assigned_fraction = 'Unknown'
    else:
        fractions_in_peak = segment['Fraction_unique_ID'].dropna().unique()

        if len(fractions_in_peak) == 0:
            assigned_fraction = 'Unknown'

```

```

elif len(fractions_in_peak) == 1:
    assigned_fraction = fractions_in_peak[0]
else:
    # Vectorize Fraction Assignment Logic
    fraction_stats = segment.groupby('Fraction_unique_ID')['
        UV_1_280_mAU'].agg(['sum', 'min'])
    fraction_stats.columns = ['sum_val', 'start_vol_frac']
    fraction_stats = fraction_stats.sort_values(by=['sum_val', '
        start_vol_frac'], ascending=[False, True])

    # Check to ensure fraction_stats is not empty before accessing
    index
    if fraction_stats.empty:
        assigned_fraction = 'Unknown'
    else:
        assigned_fraction = fraction_stats.index[0]

run_peaks_info.append({
    'run_no': run,
    'Peak_ID': f"{run}_P{idx+1}",
    'Start_Volume': start_vol,
    'End_Volume': end_vol,
    'Area_280': area_280,
    'Area_260': area_260,
    'Purity_Ratio': purity_ratio,
    'Associated_Fraction_ID': assigned_fraction
})

# Step 7: Output Handling
if len(run_peaks_info) > 15:
    run_peaks_info.sort(key=lambda x: x['Area_280'], reverse=True)
    selected_peaks = run_peaks_info[:15]
    omitted_count = len(run_peaks_info) - 15
    all_peaks_data.extend(run_peaks_info)
else:
    selected_peaks = run_peaks_info
    omitted_count = 0
    all_peaks_data.extend(run_peaks_info)

# Prepare Markdown output for this run
# Refactored Markdown generation block per feedback:
# 1. Determine #total_peaks#.
# 2. Set #max_display = 15# if #total_peaks > 15# else #total_peaks#.
# 3. Set #N = max_display#.
# 4. Construct #md_rows# with exactly #N# data rows.
# 5. Append omission note only if #total_peaks > 15#.
# 6. Remove the #if 2 + len(md_rows) > 20# safety slice entirely.

total_peaks = len(run_peaks_info)

# Determine max_display
if total_peaks > 15:
    max_display = 15
else:
    max_display = total_peaks

# Set N
N = max_display

```

```

# Slice selected_peaks to exactly N rows
# Note: selected_peaks is already sorted and sliced to 15 if total > 15 in the
#       previous block.
# But we must ensure we don't exceed N if logic changed.
if len(selected_peaks) > N:
    selected_peaks = selected_peaks[:N]

md_header = "Peak_ID,Start_Volume,End_Volume,Area_280,Area_260,Purity_Ratio,
Associated_Fraction_ID"
md_rows = [md_header]

for p in selected_peaks:
    # Feedback: Add Type Safety for Markdown Output. Ensure #
    # Associated_Fraction_ID# is explicitly converted to a string.
    fraction_id_str = str(p['Associated_Fraction_ID'])
    row = f"{p['Peak_ID']},{p['Start_Volume']:.4f},{p['End_Volume']:.4f},{p['
    Area_280']:.4f},{p['Area_260']:.4f},{p['Purity_Ratio']:.4f},{
    fraction_id_str}"
    md_rows.append(row)

# Append the omission note only if total_peaks > 15
if total_peaks > 15:
    omitted_count = total_peaks - N
    md_rows.append(f"Nadditional_peaksomitted({omitted_count})")

# Total lines check: Title(1) + Open(1) + Header(1) + N + Note(1 if >15) +
# Close(1)
# 1 + 1 + 1 + 15 + 1 + 1 = 20.
# 1 + 1 + 1 + 16 + 0 + 1 = 20.
# The logic ensures it never exceeds 20.

markdown_lines.append(f"###Run:{run}")
markdown_lines.append("###csv")
markdown_lines.extend(md_rows)
markdown_lines.append("###")
markdown_lines.append("")

# Write Markdown Output
with open(OUTPUT_MARKDOWN, 'w') as f:
    f.write("\n".join(markdown_lines))

# Write Intermediate CSV (Full structured data)
if all_peaks_data:
    df_peaks = pd.DataFrame(all_peaks_data)
    df_peaks.to_csv(OUTPUT_CSV, index=False)
else:
    pd.DataFrame(columns=['run_no', 'Peak_ID', 'Start_Volume', 'End_Volume', '
    Area_280', 'Area_260', 'Purity_Ratio', 'Associated_Fraction_ID']).to_csv(
    OUTPUT_CSV, index=False)

```

Listing 2: Code_2

```

import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import os
from scipy.signal import find_peaks

# Ensure output directory exists

```

```

os.makedirs('/workspace', exist_ok=True)

# Step 0: Identify the correct raw input file in /inputs/
inputs_dir = '/inputs'
raw_input_file = None

if not os.path.isdir(inputs_dir):
    raise FileNotFoundError(f"Inputs directory '{inputs_dir}' does not exist.")

input_files = os.listdir(inputs_dir)
csv_files = [f for f in input_files if f.endswith('.csv')]

if not csv_files:
    raise FileNotFoundError(f"No CSV files found in '{inputs_dir}'. Cannot proceed with analysis.")

# Select the first available CSV file as the raw input
raw_input_file = os.path.join(inputs_dir, csv_files[0])
print(f"Using raw input file: {raw_input_file}")

intermediate_output_file = '/workspace/step2_intermediate_peaks.csv'

if not os.path.exists(raw_input_file):
    raise FileNotFoundError(f"Raw input file '{raw_input_file}' not found. Cannot proceed with analysis.")

df_raw = pd.read_csv(raw_input_file)

# Step 1: Identify signal and x-axis columns dynamically
# Expected columns based on error context: 'UV_1_280_mAU' and 'volume_ml' or 'Fraction_number'
# Fallback logic if exact names vary slightly
signal_col = None
x_axis_col = None

if 'UV_1_280_mAU' in df_raw.columns:
    signal_col = 'UV_1_280_mAU'
elif 'UV_280' in df_raw.columns:
    signal_col = 'UV_280'
else:
    # Fallback: find column containing 'UV' and '280'
    candidates = [c for c in df_raw.columns if 'UV' in c and '280' in c]
    if candidates:
        signal_col = candidates[0]

if 'volume_ml' in df_raw.columns:
    x_axis_col = 'volume_ml'
elif 'Fraction_number' in df_raw.columns:
    x_axis_col = 'Fraction_number'
else:
    # Fallback: find column containing 'volume' or 'Fraction'
    candidates = [c for c in df_raw.columns if 'volume' in c.lower() or 'fraction' in c.lower()]
    if candidates:
        x_axis_col = candidates[0]

if not signal_col or not x_axis_col:
    raise ValueError(f"Could not identify signal or x-axis columns. Available: {df_raw.columns.tolist()}")

```



```

# Step 2: Perform Peak Detection and Integration
# Sort by x-axis to ensure correct integration order
df_sorted = df_raw.sort_values(by=x_axis_col).reset_index(drop=True)

signal_values = df_sorted[signal_col].values
x_values = df_sorted[x_axis_col].values

# Detect peaks using scipy
# height threshold can be adjusted; using 10% of max signal as a heuristic
threshold_height = np.max(signal_values) * 0.1
peaks, properties = find_peaks(signal_values, height=threshold_height, distance=5)

# Prepare lists to build the new dataframe
new_rows = []

if len(peaks) == 0:
    # If no peaks found, create an empty dataframe with required structure
    df_peaks = pd.DataFrame(columns=['run_no', 'Peak_ID', 'Area_280', 'Valid_Flag'])
else:
    # Define boundaries for integration (midpoint between peaks)
    # Add start and end indices to handle first and last peaks
    all_indices = [0] + list(peaks) + [len(signal_values) - 1]

    for i, peak_idx in enumerate(peaks):
        # Determine start and end of the peak region
        if i == 0:
            start_idx = 0
        else:
            prev_peak = peaks[i-1]
            start_idx = int((prev_peak + peak_idx) / 2)

        if i == len(peaks) - 1:
            end_idx = len(signal_values) - 1
        else:
            next_peak = peaks[i+1]
            end_idx = int((peak_idx + next_peak) / 2)

        # Extract segment for integration
        segment_signal = signal_values[start_idx:end_idx+1]
        segment_x = x_values[start_idx:end_idx+1]

        # Calculate Area using trapezoidal rule
        # Subtract baseline (min of segment) to get net peak area
        baseline = np.min(segment_signal)
        net_signal = segment_signal - baseline
        area = np.trapz(net_signal, segment_x)

        # Ensure area is positive
        area = max(0, area)

        # Assign Peak_ID (1-based index)
        peak_id = i + 1

        # Valid_Flag: 1 if area is significant (e.g., > 0), else 0
        valid_flag = 1 if area > 0 else 0

        # Determine run_no: assuming 'run_no' exists in raw data, otherwise

```

```

        default to 1
    # If raw data has run_no, we need to map it. Assuming raw data is single
    run or has column.
    # If 'run_no' is in df_raw, take the unique value or the value at the peak
    index
    if 'run_no' in df_raw.columns:
        # Get run_no corresponding to the peak index in the sorted dataframe
        # We need to map back to original index if sorting changed order, but
        here we sorted df_raw
        # So df_sorted index corresponds to df_raw index after sort.
        # Assuming run_no is constant for the file or we take the value at
        peak_idx
        run_no_val = df_sorted.loc[peak_idx, 'run_no']
    else:
        run_no_val = 1 # Default run number if not present

    new_rows.append({
        'run_no': run_no_val,
        'Peak_ID': peak_id,
        'Area_280': area,
        'Valid_Flag': valid_flag
    })

    df_peaks = pd.DataFrame(new_rows)

# Filter for valid peaks only (Step 2 Logic)
df = df_peaks[df_peaks['Valid_Flag'] == 1].copy()

# Save the intermediate file for downstream steps
df.to_csv(intermediate_output_file, index=False)
print(f"Generated intermediate file: {intermediate_output_file}")

# Step 2 (Renamed from original Step 2): Calculate Total_Valid_Area per run
if not df.empty:
    df['Total_Valid_Area'] = df.groupby('run_no')['Area_280'].transform('sum')

    # Step 3 (Renamed from original Step 3): Calculate Relative_Proportion
    # Avoid division by zero
    df['Relative_Proportion'] = df.apply(
        lambda row: row['Area_280'] / row['Total_Valid_Area'] if row['
        Total_Valid_Area'] > 0 else 0,
        axis=1
    )
else:
    df['Total_Valid_Area'] = 0
    df['Relative_Proportion'] = 0

# Step 4: Generate Comparative Analysis (Stacked Bar Chart)
unique_runs = df['run_no'].unique()
unique_peaks = df['Peak_ID'].unique()

num_runs = len(unique_runs)
num_peaks = len(unique_peaks)

# Determine grouping strategy for peaks
if num_peaks > 10 and not df.empty:
    # Vectorized Peak Categorization
    conditions = [
        df['Relative_Proportion'] > 0.1,

```

```

        (df['Relative_Proportion'] >= 0.01) & (df['Relative_Proportion'] <= 0.1)
    ]
    choices = ['Major', 'Minor']
    df['Peak_Category'] = np.select(conditions, choices, default='Trace')

    # Aggregate by run and category for plotting
    plot_data = df.groupby(['run_no', 'Peak_Category'])['Relative_Proportion'].sum()
    ().unstack(fill_value=0)
    categories = ['Major', 'Minor', 'Trace']
    colors = ['#e41a1c', '#377eb8', '#4daf4a']
    plot_labels = categories
else:
    # Use individual Peak_IDs
    if not df.empty:
        plot_data = df.groupby(['run_no', 'Peak_ID'])['Relative_Proportion'].sum()
        .unstack(fill_value=0)
        categories = unique_peaks
        colors = plt.cm.tab10(np.linspace(0, 1, num_peaks))
        plot_labels = unique_peaks
    else:
        plot_data = pd.DataFrame()
        categories = []
        colors = []
        plot_labels = []

# Ensure plot_data index is sorted to match unique_runs order
if not plot_data.empty:
    plot_data = plot_data.reindex(unique_runs)

# Setup plotting
fig = None
if not df.empty and num_runs > 0:
    if num_runs > 10:
        # Split into subplots
        cols = 2
        rows = (num_runs + cols - 1) // cols
        fig, axes = plt.subplots(rows, cols, figsize=(15, 5 * rows))
        axes = axes.flatten()

        # Plot only valid runs
        for i, run in enumerate(unique_runs):
            if i >= len(axes):
                break
            ax = axes[i]
            if run in plot_data.index:
                run_data = plot_data.loc[run]
                run_data.plot(kind='bar', stacked=True, ax=ax, color=colors)
                ax.set_title(f'Run: {run}')
                ax.set_ylabel('Relative Proportion')
                ax.set_xlabel('Category/Peak')
                ax.set_ylim(0, 1.05)

        # Clean up unused axes
        for i in range(num_runs, len(axes)):
            axes[i].remove()

    plt.tight_layout()
else:
    fig, ax = plt.subplots(figsize=(12, 6))

```

```

        if not plot_data.empty:
            plot_data.plot(kind='bar', stacked=True, ax=ax, color=colors)
            ax.set_title('Relative_Compound_Proportions_per_Run')
            ax.set_xlabel('Run_No')
            ax.set_ylabel('Relative_Proportion')
            ax.set_ylim(0, 1.05)

plt.savefig('/workspace/run_proportion_comparison.png', dpi=300, bbox_inches='
tight')
plt.close(fig)

# Step 5: Identify Outliers and Generate Summary Table
if not df.empty:
    summary_stats = df.groupby('run_no').agg(
        Total_Valid_Area=('Area_280', 'sum'),
        Number_of_Peaks=('Peak_ID', 'count'),
        Sum_of_Relative_Proportions=('Relative_Proportion', 'sum')
    ).reset_index()
else:
    summary_stats = pd.DataFrame(columns=['run_no', 'Total_Valid_Area', '
Number_of_Peaks', 'Sum_of_Relative_Proportions'])

# Flag outliers (Sum deviates significantly from 1.0, e.g., > 0.05 deviation)
if not summary_stats.empty:
    summary_stats['Is_Outlier'] = (abs(summary_stats['Sum_of_Relative_Proportions'
] - 1.0) > 0.05).astype(int)
else:
    summary_stats['Is_Outlier'] = 0

# Define header and separator
header_line = "|_run_no_|_Total_Valid_Area_|_Number_of_Peaks_|_
Sum_of_Relative_Proportions_|"
separator_line = "
|-----|-----|-----|-----|"

# Calculate max data rows based on 20 line limit
# Total lines = Title (1) + Header (1) + Separator (1) + Data Rows + Truncation
Row (1 if truncated) + Footer (1 if outliers)
max_total_lines = 20
fixed_lines = 3 # Title + Header + Separator
footer_count = 1 if not summary_stats.empty and summary_stats['Is_Outlier'].any()
else 0
# Reserve 1 line for potential truncation row
max_data_rows = max_total_lines - fixed_lines - 1 - footer_count

# Generate data rows using vectorized formatting
if not summary_stats.empty:
    data_rows = summary_stats.apply(
        lambda row: f"|_{row['run_no']}|_{row['Total_Valid_Area']:.4f}|_{row['
Number_of_Peaks']}|_{row['Sum_of_Relative_Proportions']:.4f}|_",
        axis=1
    ).tolist()
else:
    data_rows = []

# Truncate if necessary
truncated = False
if len(data_rows) > max_data_rows:
    data_rows = data_rows[:max_data_rows]

```

```

        truncated = True

# Construct final table content
md_table_content = f"###_Run-Level_Normalization_Summary\n\n{header_line}\n{
    separator_line}\n"
md_table_content += "\n".join(data_rows)
if truncated:
    md_table_content += "\n|_|..._|_|..._|_|..._|_|..._|"

# Add footer if outliers exist
if not summary_stats.empty and summary_stats['Is_Outlier'].any():
    md_table_content += "\n\n**Outliers_Detected:**_Runs_with_significant_
        deviation_in_sum_of_proportions."

# Append to final_summary.md
summary_file = '/workspace/final_summary.md'
if os.path.exists(summary_file):
    with open(summary_file, 'a') as f:
        f.write("\n")
        f.write(md_table_content)
else:
    with open(summary_file, 'w') as f:
        f.write("#_Final_Analysis_Summary\n\n")
        f.write(md_table_content)

print("Analysis_complete._Outputs_saved_to_/workspace/")

```

Listing 3: Code_3